AEGIS.

TRUST MESH CONSOLE

Bounded. Revocable. Verifiable.

Final Recording Playbook

RAZORPAY BUILDATHON — OFFICIAL RECORDING OPERATIONS MANUAL

"What happens when an AI can act with financial authority — and why trust cannot depend on trusting the AI itself."

v1.0 · Verified against the live running application before writing

Table of Contents

PART 1

1. The Mission — What We Are Actually Recording

What we are recording: a 7–8 minute video submission for the Razorpay Buildathon — not a live presentation, not a Q&A, not a tutorial. Nobody watching can ask a question or wait for a page to load. Everything on screen has to already be exactly what it needs to be the instant the camera reaches it.

What we are not doing: clicking through six tabs in order and narrating what each button does. That is a UI walkthrough, and a judge who has watched twenty of those today will not remember it in an hour.

What we are doing instead

We are telling one story, and every workspace shows up inside that story only when the story needs it:

```
An AI agent can now act with financial authority.
↓
If authority alone decided everything, trust would have to be blind.
↓
AEGIS makes authority bounded, revocable, and verifiable — instead.
↓
Every claim in that sentence is proven live, on screen, not asserted.
```

The AEGIS thesis

> Bounded. Revocable. Verifiable.

What the judges should understand by the end

- Authority in AEGIS is a cryptographic capability token, not a config flag — and it can only ever narrow as it's delegated, never widen.
- Spending is bound to a specific, budgeted mission — and that budget genuinely survives real concurrent load, not just sequential testing.
- Every transaction resolves to one of three real outcomes — allow, deny, or escalate — through a pipeline that never lets a probabilistic AI judgment override a deterministic boundary.
- Authority can be revoked instantly, and revocation is a live, continuously-checked relationship, not a one-time grant.
- Every decision is recorded in a signed, hash-chained ledger, and a completely separate tool — sharing no code with AEGIS itself — can independently prove whether that history was tampered with.

If a judge remembers nothing else, they should remember this: **AEGIS does not ask anyone to trust the AI. It makes the boundary around the AI enforceable and provable.**

PART 2

2. Absolute Start-to-Finish Checklist

STEP 0 — Before touching the application

- ☐ Laptop plugged in (a mid-recording low-battery warning is not recoverable mid-take)
- ☐ Stable network connection — not strictly required by the application (demo mode makes zero external calls), but required for screen-recording software, browser stability, and avoiding OS-level connectivity popups
- ☐ OS notifications disabled / Do Not Disturb enabled
- ☐ Browser: all unrelated tabs closed, only the AEGIS tab (and a terminal tab, if using the verifier CLI segment) open
- ☐ Screen resolution and browser window sized to 1920×1080 (or your recording software's target canvas), browser zoom confirmed at 100%
- ☐ Recording software open, target window/region selected, a 5-second test capture reviewed
- ☐ Microphone tested — record 10 seconds, play it back, confirm level and no background hum
- ☐ Desktop background and taskbar clean of anything embarrassing or identifying
- ☐ No API keys, tokens, or secrets visible anywhere — no open `.env`, no terminal scrollback showing a real key
- ☐ No terminal window visible unless it's the one you're deliberately using for the verifier CLI segment, and even then, scrolled clean first

STEP 1 — Start the AEGIS server

Verified this session, directly against `package.json` and a live boot. Run from the repository root (`D:\Desktop\Aegis`).

Terminal location

Repository root — the same directory that contains `package.json`.

Exact command

```
AEGIS_DEMO_MODE=true AEGIS_DB_PATH=./aegis-recording.db PORT=8787 npm start
```

This is `npm run build && node --experimental-wasm-modules dist/api/main.js` under the hood (see `package.json`'s `start` script) — it rebuilds first, so the very first run after any change takes a few seconds longer. Using a dedicated `AEGIS_DB_PATH` here (rather than the default `./aegis.db`) guarantees a database with no leftover history from any earlier session.

Expected output

```
================================================================================
  AEGIS — LOCAL DEMO MODE (AEGIS_DEMO_MODE=true)
  Intent judge : deterministic stand-in — NOT a real AI risk evaluation
  Rails        : mock_x402 ONLY — Stripe is never registered in this mode
  No real money can move. This proves the SOFTWARE PIPELINE, not payment.
================================================================================
...
Mock x402 demo merchant listening at http://127.0.0.1:<port> — fixed price catalog: ...
AEGIS_DEMO_MODE is enabled — the stripe_test rail is never registered in demo mode, regardless
Aegis listening on http://localhost:8787
Dashboard: http://localhost:8787/
```

How to verify the server is healthy

In a second terminal: `curl http://localhost:8787/demo-mode` → must return `{"demoMode":true}` with HTTP 200. If this fails, the server did not start correctly — check the first terminal for an error before doing anything else.

Exact URL to open

`http://localhost:8787`

STEP 2 — Verify clean starting state

CHECK	HOW	WHAT "CLEAN" LOOKS LIKE
Server health	`curl http://localhost:8787/demo-mode`	`{"demoMode":true}`, HTTP 200
Demo mode active	Same check	`demoMode: true` — confirms no real rail, no external AI call possible
No accidental tampering	Sign in → Evidence tab → the header status next to your principal name	Should read **"Ledger verified"**, never **"Ledger tampered"**. If it already says tampered, this database was used for a rehearsal that included the tamper step — restart with a fresh `AEGIS_DB_PATH` before recording.
Correct initial UI state	Overview tab	Fresh principal shows "No agent selected," "No mission selected," "No transaction submitted yet," Ledger **"Verified · 0"**

WHEN A DATABASE RESET IS NECESSARY

Any time the Evidence tab already shows "Ledger tampered," or the concurrent-race attack mission has already been spent down from a prior rehearsal and you don't want that visible in the opening state.

WHEN A RESET IS NOT NECESSARY

Between most segments — creating a new agent, a new mission, or a new principal does not require restarting the server. Signing in with a fresh principal ID alone gives you a clean Authority/Missions/Transactions slate without touching anything else. Only the tamper step is a genuine one-way door (see Part 3D).

PART 3

3. 🔥 Pre-Staging — Do This Before Recording

The recording must never depend on figuring something out while the camera is running. Everything below is prepared *before* you press record, in a rehearsal pass, so that on the real take only the specific click sequence in Part 5 happens on camera.

A. ESCALATE baseline pre-staging

Why ESCALATE cannot be demonstrated from a completely untouched agent: the escalate path is triggered by `src/risk/baseline.ts` 's amount-deviation check, which needs at least 3 prior transactions on that specific agent before it has a "historical average" to compare against. A brand-new agent's very first transaction has no history to deviate from, so it cannot escalate on that basis — the baseline must genuinely exist first.

Exact setup

1. Create a dedicated agent for this scenario only — do not reuse the ALLOW/DENY agent. Suggested ID: `escalate-agent` . Max amount: `5000` . Categories: `software` . Do not attach a mission — use standing authority (leave the transaction form's Mission field as "None").
2. Go to Transactions, with this agent selected.
3. Submit **three** baseline transactions, one at a time, waiting for each verdict before submitting the next:
   • Amount `50` · Category `software` · Rail `mock_x402` · Counterparty `cloudco` · Purpose "Small recurring software license"
   • Repeat two more times, same values (the exact purpose text can vary slightly, the amount must stay `50`)
4. Confirm each of the three returns **ALLOW** before moving to the next.

PRE-STAGE CHECKLIST — ESCALATE

- ☐ Baseline transaction 1 completed ($50, ALLOW)
- ☐ Baseline transaction 2 completed ($50, ALLOW)
- ☐ Baseline transaction 3 completed ($50, ALLOW)
- ☐ History verified — three green ALLOW entries visible for this agent
- ☐ Escalation transaction (the $600 one) **NOT** submitted yet
- ☐ Recording can now begin

The exact final transaction that triggers ESCALATE

Amount `600` · Category `software` · Rail `mock_x402` · Counterparty `cloudco` · same agent, no mission. This is 12× the $50 baseline mean, well over the 3× threshold. Verified result this session: verdict **ESCALATE**, reason text "*Behavioral anomaly detected: Amount (60000) is 12.0x this agent's historical average (5000), over the 3x threshold*" (amounts shown in minor units — 60000 = $600.00, 5000 = $50.00), with the `Intent:` line still reading *consistent*.

WHAT THE VIEWER SHOULD AND SHOULD NOT SEE

The recording should show only the **final $600 transaction** causing the escalation. The three $50 baseline transactions are preparation, not content — they happen before you press record, exactly like a magician's setup happens before the curtain rises. Do not narrate or show the baseline-building on camera; it adds nothing and burns 30+ seconds of dead time.

B. Concurrent budget attack pre-staging

Preparation

1. Select any agent with a token cap of at least $2,000 (the pre-existing root agent from the Authority segment works).
2. Go to Security → Concurrent Budget Race.
3. Click **New attack mission ($2,000)** — this is a fixed amount built into the scenario; it always creates a fresh, untouched $2,000 mission.
4. Click **Launch attack (20 × $380)** — fires 20 genuinely concurrent `POST /transactions` calls at $380 each against that one mission.

WHAT IS GUARANTEED	WHAT MAY VARY
The mission budget is **never** exceeded — `allowed × 380 ≤ 2000`, always. The banner always reads **"✓ ZERO OVERSPEND — BUDGET HELD"**, with a server-confirmed spend figure underneath.	The exact split of how many of the 20 attempts land as allowed vs. blocked. This session's own verified run produced 5 allowed / 15 blocked / $1,900 spent — but this depends on real request-scheduling timing, not a script, and can differ between runs.

DO NOT MEMORIZE THE NUMBER OF ALLOWED REQUESTS

Say whatever number actually appears on your take. The only fixed claim to make out loud is the invariant: "*the mission budget is never exceeded, no matter how the twenty requests race.*" A natural way to phrase it live: "*Twenty requests just competed for the same budget — [X] allowed, [Y] blocked, and the total spent never crossed the line.*"

C. Revocation pre-staging

What must exist before recording

Nothing needs to be manually pre-staged. The Security workspace's Delegation & Revocation panel builds its own fresh parent/child agent pair and runs the entire before/after sequence internally, every time you click **Run scenario**. This is by design — it's idempotent, safe to run once per take with no setup.

Exact action and expected result

Security tab → click **Run scenario**. It renders, in order: a "Before revocation" line showing **ALLOW** ("*Policy satisfied; consistent with delegated goal; no behavioral anomalies*"), then an "After revocation" box showing a large red ✗ **DENY** with "*Token (or an ancestor) was revoked at <timestamp>: Attack-theatre demo revocation*" and "*Rail calls after revocation: 0.*"

USE THIS SCENARIO — NOT THE AUTHORITY TAB'S MANUAL REVOKE

Clicking Revoke directly on an agent card in the Authority tab works correctly on the backend, but that card gives **no visible confirmation** on screen afterward (still shows Select/Attenuate/Revoke, unchanged). It would be a confusing, undramatic beat on camera. The Security workspace's scenario is purpose-built for this exact demonstration and is the only revocation path that should appear in the video.

D. Evidence / tamper detection pre-staging

Exact sequence

1. Evidence tab → click **Refresh**, then **Verify chain** → confirm green **"✓ HASH CHAIN VERIFIED."**
2. Click **Tamper latest entry** — this corrupts one stored ledger entry directly in the database, bypassing AEGIS's own write path entirely.
3. Click **Verify chain** again → confirm red **"✗ INTEGRITY VIOLATION DETECTED,"** with the exact corrupted entry named underneath (verified this session: "*Entry #103 (agent_registered) was altered directly in storage, bypassing Aegis entirely…*" — the exact entry number will differ depending on how much ledger history exists by the time you reach this step).

⚠ RECORD THIS SEGMENT LAST

Tampering the ledger is a genuine one-way state change for the running server process — there is no "un-tamper" button. Every segment recorded after this one will show a header badge reading "Ledger tampered," which is fine if this really is your last app segment, but wrong if you still need to show ALLOW/DENY/ESCALATE or the concurrent race afterward.

If a retake of this segment is needed

Stop the server, delete `aegis-recording.db` (and any `-journal` / `-wal` / `-shm` siblings), restart with the exact Step 1 command, re-verify `/demo-mode`, and re-stage whichever earlier segments this retake needs to lead into (e.g. re-create the root/child agents if the video's Authority segment needs to be re-shot too). If only the tamper beat itself needs a retake and everything before it in the final edit is already good footage, you only need a fresh database — the earlier segments' footage doesn't need to be re-recorded.

PART 4

4. Exact Recording Order

#	SEGMENT	TIME	WHAT TO SHOW	STATE REQUIRED
1	Hook	00:00–00:35	Voice-over, no UI	None
2	AEGIS reveal	00:35–01:05	Overview hero statement	Signed in, clean principal
3	Authority + attenuation	01:05–01:55	Live attenuate + rejected widen attempt	Root agent pre-staged ($2,000)
4	Mission boundaries	01:55–02:35	Mission create + Detail budget math	Child agent from step 3
5	Transaction pipeline: ALLOW	02:35–03:45	$380 flights, green pipeline	ALLOW-agent + $800 mission, untouched
6	DENY		$5,000 over an $800 mission	Same agent/mission as above
7	ESCALATE		$600 vs. $50×3 baseline	Escalate-agent, baseline already staged (Part 3A)
8	Concurrent attack	03:45–04:40	20×$380 vs $2,000, zero overspend	Fresh attack mission clicked immediately before (Part 3B)
9	Revocation	04:40–05:20	Before/after, one click	None — self-contained scenario
10	Evidence verification	05:20–06:25	Verify chain — green	Ledger currently clean
11	Tamper detection		Tamper → verify → red violation	**Record last of all app segments**
12	Architecture / credibility	06:25–07:00	Test-count montage, demo-mode honesty line	Pre-captured `npm test` output
13	Closing statement	07:00–07:40	Back to Overview hero shot	None

PART 5 — THE MOST IMPORTANT SECTION

5. The Full 7–8 Minute Script

Target 7:35, inside the 7:00–8:00 requirement. Every quoted UI string below is a real string this application produces — verified live this session, not paraphrased.

⏱ 00:00–00:35 — The Hook

WHAT I SAY

"Give an AI agent the ability to spend money, and it can book your flights, pay your vendors, renew your subscriptions — faster than anyone on your team ever could.

Give it the ability to delegate that spending to another agent it spins up on its own, and the picture changes.

Because now the question isn't whether an AI can spend money. It already can.

It's: what, exactly, is it allowed to do — who gave it that authority — and if something goes wrong, can you cut it off, instantly, and prove afterward exactly what happened?"

WHAT I DO

Nothing — voice-over only, do not touch the keyboard.

WHAT IS ON SCREEN

Black, or the AEGIS wordmark held on a dark ground. No dashboard yet.

WHAT I POINT AT

Nothing — the tension is entirely spoken.

WHY THIS MATTERS

Opens on the actual danger, not a self-introduction — the judge leans in before seeing a single click.

⏱ 00:35–01:05 — The Reveal

WHAT I SAY

"That's the problem AEGIS was built to solve.

AEGIS doesn't ask you to trust an autonomous agent. It gives that agent authority — with boundaries.

Bounded. Revocable. Verifiable."

WHAT I DO

Cut to the live Overview workspace, already signed in — no loading visible.

WHAT IS ON SCREEN

The hero statement, timed to land as you say the words.

WHAT I POINT AT

The environment photograph behind the UI — let it register a beat. First proof this is a real product, not a slide.

WHY THIS MATTERS

Names the thesis once, cleanly, before any UI complexity begins.

⏱ 01:05–01:55 — Authority, Delegation, Attenuation

WHAT I SAY

"Authority in AEGIS isn't a permission flag in a database. It's a signed cryptographic token — a maximum amount, allowed categories, allowed payment rails, all baked into the credential itself.

Here's where most permission systems fail: they can grant access, but they can't safely delegate it. AEGIS can.

Watch what happens when I delegate this agent's authority to a narrower sub-agent. The child can only ever get narrower than its parent — never wider. If I try to hand it more than the parent has —" [attempt to widen] "— it's refused. Not by a UI validation rule. By the token's own math."

WHAT I DO

Root agent's **Attenuate** → fill `child-agent` / `$800` / `flights` → **Create**. Then **Attenuate** again on the child → `$5000` + category `crypto` → **Create**.

WHAT IS ON SCREEN

Parent card `cap 2000.00 USD · flights,hotels,software`; child card `cap 800.00 USD · flights`; then the real inline error: "*Attenuation error: child maxAmountMinorUnits (500000) exceeds parent's (80000).*"

WHAT I POINT AT

The two `.caveats` lines side by side, then the red error text.

WHY THIS MATTERS

The single strongest technical-credibility moment in the video — a real constraint engine refusing a real attempt, not a form validator.

⏱ 01:55–02:35 — The Mission

WHAT I SAY

"Authority answers what this agent could ever do. A mission answers what it's allowed to do right now.

I'm binding this agent to one bounded objective — an $800 budget, flights only, one approved vendor. The agent doesn't get money. It gets permission to complete this one mission, inside boundaries no wider than its own authority allows."

WHAT I DO

Missions tab → create: budget `800`, category `flights`, counterparty `acme-airlines` → open Mission Detail.

WHAT IS ON SCREEN

"*Budget — the boundary this mission cannot exceed*" and the live arithmetic `800.00 USD budget − 0.00 USD settled = 800.00 USD remaining.`

WHAT I POINT AT

That live subtraction — it updates from real state, not a static label.

WHY THIS MATTERS

Establishes the authority-vs-mission distinction that the next segment's DENY depends on.

⏱ 02:35–03:45 — The Pipeline: ALLOW, DENY, ESCALATE

WHAT I SAY

"Every transaction runs through a real decision pipeline before anything moves — mission budget, capability check, risk evaluation. Let's put three real transactions through it.

First — a valid one. $380, a flight, within every boundary." [Execute] "Allow. Every stage green, real settlement."

WHAT I DO

Amount `380` / `flights` / `mock_x402` / `acme-airlines`, mission attached → **Execute**.

WHAT IS ON SCREEN

Green `ALLOW` badge, every pipeline stage lit green, a real settlement reference.

WHAT I POINT AT

The stage-by-stage trace — nothing skipped.

WHY THIS MATTERS

Establishes the baseline "everything working" case before showing the two that stop.

WHAT I SAY

"Now — the same agent, wildly over its mission's budget." [Execute $5,000] "Deny. Not a warning. Nothing executes. And notice exactly where it stopped — the mission gate, before the pipeline even reaches the capability check."

WHAT I DO

Change amount to `5000`, same mission → **Execute**.

WHAT IS ON SCREEN

Red `DENY`, mission-gate stage highlighted, reason: *"Transaction would exceed this mission's budget…"*

WHAT I POINT AT

Exactly which stage stopped it — nothing after it rendered as having run.

WHY THIS MATTERS

Proves a denial is a hard stop with a named cause, not a generic rejection.

WHAT I SAY

"And here's the one people don't expect. This agent has a small, consistent spending pattern. I'm about to submit something twelve times its normal size." [Execute $600] "Escalate. Not denied — because no hard rule was broken. Flagged, because the behavior doesn't look like this agent anymore."

WHAT I DO

Switch to the pre-staged escalate-agent (Part 3A) → Amount `600` → **Execute**.

WHAT IS ON SCREEN

Gold `ESCALATE` badge, reason: *"Behavioral anomaly detected: Amount (60000) is 12.0x this agent's historical average (5000), over the 3x threshold"*, with `Intent: consistent` shown right beside it.

WHAT I POINT AT

The `Intent: consistent` line specifically — *"the AI judge here isn't even the reason this got flagged — a separate, purely mathematical signal is."*

WHY THIS MATTERS

The single most technically credible line in the video — the safety boundary was never handed to a probabilistic model.

⏱ 03:45–04:40 — The Engineering Challenge: Concurrent Budget Race

WHAT I SAY

"Here's a real problem we had to solve building this. A mission has a budget. Fine — check the amount against it before letting a transaction through.

But autonomous agents don't send one request at a time. What happens when twenty requests hit the same $2,000 budget, at the same instant, each asking for $380? That's $7,600 of exposure against a $2,000 limit.

If each request checks the balance, sees it's fine, and only then writes its spend — they can all check at once, all see room, and all proceed. That's a real race condition, and it's exactly the kind of bug that never shows up until you're actually under load.

AEGIS closes that gap with one atomic database write that checks and reserves budget in the same instant — so it's not a matter of ordering, it's a matter of arithmetic. Watch what happens when I fire all twenty at once." [Launch attack] "[X] allowed. [Y] blocked. [Z] spent. Zero overspend. The budget held — under real concurrency, not a best case."

WHAT I DO

New attack mission ($2,000) → **Launch attack (20 × $380)** → let it resolve on camera (~4–5s).

WHAT IS ON SCREEN

Live per-attempt grid resolving, then the stat row and the green **"✓ ZERO OVERSPEND — BUDGET HELD"** banner.

WHAT I POINT AT

The server-confirmed spend line under the banner — the receipts, not just a UI claim.

WHY THIS MATTERS

Most hackathon demos never touch concurrency at all. This is the moment that separates a working system from a UI mockup — **state the numbers you actually see on screen, not memorized ones (see Part 3B).**

⏱ 04:40–05:20 — The Emergency Brake: Revocation

WHAT I SAY

"Authority isn't a permanent credential. It's a live relationship, checked fresh on every single transaction. Watch.

This transaction works, right now." [before] "Allow.

Now I revoke it." [Run scenario] "And the exact same request —" [after] "Deny.

The transaction didn't change. The authority did."

WHAT I DO

Security tab → click **Run scenario** — one click runs the entire before/after sequence.

WHAT IS ON SCREEN

Green `ALLOW … Policy satisfied…`, then a large red `✗ DENY` box: *"Token (or an ancestor) was revoked at …: Attack-theatre demo revocation"* and *"Rail calls after revocation: 0."*

WHAT I POINT AT

"Rail calls after revocation: 0" — the denial isn't cosmetic, nothing downstream ever got called.

WHY THIS MATTERS

Short, clean, conceptually powerful — a good pacing anchor right after the denser concurrency segment.

⏱ 05:20–06:25 — The Proof: Evidence, Tamper Detection, Independent Verification

WHAT I SAY

"But enforcement only matters if you can prove it happened. A dashboard telling you a decision was made is not evidence — it's a claim.

Every decision AEGIS makes is written to a signed, hash-chained ledger — each entry linked to the one before it. Let's verify it." [Verify chain] "Hash chain verified.

Now watch — I'm going to tamper with a stored entry directly in the database. Bypassing AEGIS entirely, the way someone with raw storage access would." [Tamper → Verify again] "Integrity violation detected. Not a guess — it names the exact entry, because its stored hash no longer matches its content.

And this doesn't require trusting AEGIS's own check. A completely separate tool — that shares zero code with this dashboard — reaches the same conclusion, offline."

WHAT I DO

Refresh → Verify chain → Tamper latest entry → Verify chain again. (Optional: pre-typed terminal command for the verifier CLI.)

WHAT IS ON SCREEN

Green `✓ HASH CHAIN VERIFIED` → red `✗ INTEGRITY VIOLATION DETECTED`, named entry, header badge flips to **"Ledger tampered."**

WHAT I POINT AT

The status flip itself — same button, same screen, green to red, live.

WHY THIS MATTERS

The emotional peak of the video. **Record last of all app segments — see Part 3D.**

⏱ 06:25–07:00 — Why This Is Real

WHAT I SAY

"None of this is scripted for the camera. It's 434 automated tests on the core pipeline, plus 58 more on that independent verifier — all passing, all reproducible.

Right now, this is running in local demo mode — a disclosed, deterministic risk judge, no external API, no real money, so this recording never depends on network access or a paid credential. The same interface has a real Anthropic-backed judge behind it for a live deployment — but we'd rather show you something honest and reproducible than fake a model call for a demo."

WHAT I DO

Pre-captured footage/screenshots only — do not run tests live on camera.

WHAT IS ON SCREEN

The Decision Inspector's own disclosure text, then a terminal flash of both suites' final summary lines.

WHAT I POINT AT

The word "deterministic" and the exact pass counts.

WHY THIS MATTERS

Precision reads as credibility — and this is the honest, complete answer to "is AI really deciding this."

⏱ 07:00–07:40 — The Close

WHAT I SAY

"The future problem was never whether an AI agent can spend money. It already can.

The real question is what it's allowed to do — and whether you can prove it stayed inside those lines.

The answer can't be: trust it.

It has to be authority that's bounded, that's revocable, and that's verifiable.

That's AEGIS."

WHAT I DO

Cut back to Overview, hold the final frame 2 seconds before cutting to black.

WHAT IS ON SCREEN

The hero statement, mirroring the opening reveal shot exactly.

WHAT I POINT AT

Nothing — let the words land.

WHY THIS MATTERS

Directly answers the question the hook opened with — no "thank you," no generic sign-off.

Total spoken runtime at this pacing: ≈ 7:35 — inside the 7:00–8:00 target.

PART 6

6. Feature Coverage Matrix

FEATURE	MENTIONED VERBALLY	SHOWN VISUALLY	DEMONSTRATED	PROOF SHOWN
Authority / capability token	✅	✅	✅	Caveats line, live creation
Attenuation (delegation narrowing)	✅	✅	✅	Live rejected-widening error
Missions / budget boundary	✅	✅	✅	Live create + Detail arithmetic
Transaction decision pipeline	✅	✅	✅	Stage-by-stage trace, 3 live verdicts
ALLOW	✅	✅	✅	Live, real settlement reference
DENY	✅	✅	✅	Live, exact stage + reason named
ESCALATE	✅	✅	✅	Live, exact reason text + Intent line
Behavioral / risk history	✅	✅	✅	Escalate reason names the deviation math
Concurrent budget protection	✅ — hero	✅	✅	Live 20-request attack, real numbers, zero-overspend banner
Revocation + ancestor cascade	✅	✅	✅	Live before/after, real denial reason
Evidence ledger (seq/hash/sig)	✅	✅	✅	Live verify, real entry data
Tamper detection	✅ — hero	✅	✅	Live tamper → re-verify, named entry
Independent verifier	✅	🟡 optional	🟡 optional	Terminal exit code, time-permitting
Persistence / auditability	✅ (folded into evidence beat)	✅	✅	Ledger entries surviving refresh
Architecture / test credibility	✅	✅	🟡 pre-captured	434/58 pass counts

FEATURE	MENTIONED VERBALLY	SHOWN VISUALLY	DEMONSTRATED	PROOF SHOWN
Honest AI / demo-mode disclosure	✅	✅	N/A	The judge's own on-screen disclosure text
Six-workspace UI / environment art	Never named directly	✅ ambient throughout	N/A	Product feel, not a technical claim
Mission expiry / cancellation	⚪ omitted	⚪	⚪	Real, not a headline beat for 8 minutes
Idempotency / crash safety	⚪ omitted	⚪	⚪	Internal; not visually demonstrable in this format
Stripe test-mode rail	⚪ omitted	⚪	⚪	Never active in demo mode by design — would need its own caveat to explain

No implemented, judge-relevant feature is claimed without corresponding on-screen proof. The three ⚪ omissions are deliberate time-budget decisions, not gaps in the product.

PART 7

7. ⭐ The Features We Must Explain Exceptionally Well

FEATURE	WHAT MOST TEAMS WOULD SAY	WHAT WE DEMONSTRATE
Attenuation	"Agents can delegate permissions to sub-agents."	A live attempt to delegate *wider* authority, refused by the token's own math, with the actual error string on screen.
Bounded missions	"You can set a spending limit."	The distinction between a standing capability ceiling and a bounded, cumulative mission budget — two independent, simultaneously-enforced boundaries — shown as live arithmetic, not a static number.
Concurrency-safe budget enforcement	"We prevent overspending."	Twenty real, genuinely concurrent requests fired at one budget, resolving live, with a server-confirmed zero-overspend figure — a property most teams never load-test at all.
Revocation	"You can revoke access."	The exact same request, before and after, flipping from ALLOW to DENY in front of the viewer — not a described capability, a proven state change.
Tamper-evident evidence	"We log everything."	A live tamper of stored data, bypassing the application's own write path entirely, and the status flipping red — audit trail as a checkable claim, not a trusted log.
Independent verification	"Our system is secure."	A second, separately-authored tool that imports zero code from AEGIS itself, reaching the same conclusion offline — a rare, genuinely strong trust property.

THE ONE SENTENCE TO INTERNALIZE

AEGIS is not *"an AI that decides whether to spend money."* It is a system that ensures that even when an AI is genuinely powerful, its authority remains **bounded, revocable, and independently verifiable** — regardless of what the AI itself decides to do.

PART 8

8. The Engineering Story — What Could Break, and How AEGIS Prevents It

Only real, currently-supported problems — nothing invented for narrative effect.

Challenge 1 — The concurrent mission budget problem

THE PROBLEM

A mission has a $2,000 budget. A naive implementation checks the remaining balance, sees it's fine, and then writes the spend — two separate steps. That's fine for one request at a time.

WHY IT MATTERED

Autonomous agents don't operate one request at a time. Twenty simultaneous $380 attempts against a $2,000 budget represent $7,600 of exposure. If each request independently checks the balance before any of them commits their spend, every one of them can observe the same "budget available" state and all proceed — a textbook check-then-write race condition, invisible in sequential testing and real under load.

WHAT A NAIVE IMPLEMENTATION WOULD DO

`if (spent + requested <= budget) { spend(); }` — correct in isolation, unsafe under concurrency, because the read and the write are two separate moments a second request can land inside of.

WHAT AEGIS DOES INSTEAD

`src/mission/reservation.ts`'s `reserve()` is a single atomic SQL `UPDATE` that computes `budget - reserved - settled ≥ requested` and increments the reservation in the same write — there is no separate read step for a second request to race against. The database's own transaction isolation decides who wins, not request ordering.

HOW WE PROVE IT

The Security workspace's live 20-request attack, plus an adversarial automated test (`src/mission/__tests__/mission-reservation.test.ts`) that fires genuinely concurrent calls and asserts on final server state. Say it like this: *"We realized a simple budget check was not enough — under concurrent requests, multiple transactions could observe the same remaining balance. So enforcement is built around the actual invariant: no combination of concurrent requests may overspend the mission budget."*

Challenge 2 — The trust/evidence problem

THE PROBLEM

A dashboard saying a decision happened is not proof that it happened, or that it hasn't been altered since. If historical records can be edited invisibly, the evidence is only as trustworthy as whoever is currently allowed to write to the database.

WHY IT MATTERED

An audit trail that can be silently rewritten is not an audit trail — it's a claim. For a system whose entire pitch is provable boundaries, self-reported evidence would be a real credibility gap.

WHAT A NAIVE IMPLEMENTATION WOULD DO

Write plain log rows to a table and trust that nobody with database access ever edits them.

WHAT AEGIS DOES INSTEAD

Every ledger entry is Ed25519-signed and hash-chained to the entry before it. A retroactive edit breaks a verifiable mathematical relationship, not just "looks suspicious." And critically, verification doesn't have to trust AEGIS's own check — a completely separate tool (`verifier/`) that imports zero code from the main application re-derives integrity from only an exported file and the public key.

HOW WE PROVE IT

Live: verify (green) → tamper a stored entry directly, bypassing the app entirely → verify again (red, exact entry named). Say it like this: *"An audit trail is not useful if it can be rewritten without anyone knowing. So the question isn't whether our dashboard says the history is intact — it's whether a tool that trusts nothing about us agrees."*

PART 9

9. Manual Recording Rehearsal Plan

REHEARSAL	WHAT TO DO	GOAL
1 — Silent UI rehearsal	Click through the entire Part 4 order with no speaking at all, including all pre-staging.	Confirm every click, field, and button still matches this document exactly — the UI is the source of truth, not this script.
2 — Spoken rehearsal	Read the Part 5 script aloud while operating the UI, at a natural pace, no camera.	Find any line that doesn't sync naturally with the on-screen action; adjust wording, not the UI.
3 — Timed rehearsal	Same as above, with a stopwatch, no stopping for mistakes.	Confirm total runtime lands in 7:00–8:00 (target 7:35). Use the timing checkpoints below.
4 — Final camera take	Record for real, following the runbook in Part 4 exactly, pre-staging already done.	Ship it.

Timing checkpoints — where you should be in the story

1:00	Should be mid-Authority segment — attenuation error already shown or about to appear.
2:00	Should be finishing the Mission segment, about to open Transactions.
3:00	Should be mid-pipeline segment — ALLOW shown, DENY about to fire or just fired.
4:00	Should be finishing ESCALATE, about to switch to Security for the concurrent race.
5:00	Should be finishing the concurrent race or already into revocation.
6:00	Should be mid-Evidence segment — verify shown, tamper about to happen or just happened.
7:00	Should be entering the architecture-credibility montage or already at the closing line.

If you are more than ~30 seconds behind any checkpoint, tighten the next segment's dialogue rather than rushing the clicks — a rushed click sequence looks worse on camera than a slightly faster voice-over.

PART 10

10. Retake / Failure Recovery

PROBLEM	WHAT HAPPENED	RETAKE IMMEDIATELY?	RESET REQUIRED?	EXACT RECOVERY
Misspeaking	Stumbled on a line	Yes	No	Pause, re-say the sentence from its start; trim in editing.
Wrong click	Clicked the wrong tab/button	Yes	Usually no	Navigate back to the correct state and re-do the beat; cut the misclick in editing.
Server error	Unexpected 500 or crash	No	Yes	Check the terminal for the error, restart per Step 1, re-verify `/demo-mode`, re-stage.
ESCALATE baseline consumed	Accidentally submitted the $600 transaction before recording, or reused an agent that already has unrelated history	No	Partial	Switch to a fresh agent, re-submit the 3× $50 baseline (Part 3A), retry.
Concurrent scenario "weird" split	Split looks unbalanced (e.g. 1 allowed / 19 blocked)	No — this is not a failure	No	Say the numbers that actually appear; the zero-overspend claim is the only fixed one. Re-run with **New attack mission** only if you personally want a more "photogenic" split — not required.
Revocation state confusion	Used the Authority tab's manual Revoke instead of the Security scenario	Yes	No	Re-do the beat using Security → **Run scenario** only.
Evidence already tampered	A rehearsal tamper wasn't reset before this take	No	Yes	Fresh server restart on a new `AEGIS_DB_PATH`, re-stage.

PROBLEM	WHAT HAPPENED	RETAKE IMMEDIATELY?	RESET REQUIRED?	EXACT RECOVERY
Accidental database state change	Something unexpected happened to agent/mission state mid-take	Depends	Depends	Check Overview's fact grid to see current state; if it's confusing on camera, restart clean rather than narrate around it.

PART 11 — PRINT THIS PAGE

11. Final Pre-Record Checklist

TECHNICAL

- ☐ Server running (`AEGIS_DEMO_MODE=true AEGIS_DB_PATH=./aegis-recording.db PORT=8787 npm start`)
- ☐ `curl http://localhost:8787/demo-mode` → `{"demoMode":true}`
- ☐ Browser open at `http://localhost:8787` , zoom 100%, fullscreen/1920×1080
- ☐ Recording software ready, test capture reviewed
- ☐ Microphone tested

APPLICATION STATE

- ☐ Root agent staged ($2,000 · flights,hotels,software)
- ☐ ALLOW/DENY agent + mission staged ($800 budget, untouched)
- ☐ ESCALATE baseline staged — 3× $50 already submitted, $600 **not yet** submitted
- ☐ Concurrent-race attack mission — will be created fresh, live, during the take (do not pre-spend it)
- ☐ Revocation — no pre-staging needed, scenario is self-contained
- ☐ Evidence — currently clean, "Ledger verified," not yet tampered
- ☐ Tamper segment reserved for absolute last

VIDEO

- ☐ This playbook (or Part 5's script + Part 11 cheat sheet) open on a second screen or printed
- ☐ Notifications disabled, Do Not Disturb on
- ☐ No secrets visible anywhere on screen
- ☐ Unrelated windows/tabs closed
- ☐ Team ready, quiet room confirmed

PART 12

12. After Recording — Before You Submit

CHECK	WHAT TO CONFIRM
Duration	Between 7:00 and 8:00 total — trim from the architecture montage (Part 5, 06:25–07:00) first if over, since it's the most compressible segment.
Every major feature shown	Cross-check against Part 6's matrix — every ✅ row should actually appear in the final edit.
Audio	Understandable throughout, no clipping, consistent level across cuts from different takes.
UI readable	Text legible at the video's final export resolution, not just on your recording monitor.
No secrets exposed	Scrub every frame that shows a terminal — no real API keys, no `.env` contents, no signing key hex strings.
No false claims	Nothing implies Razorpay integration, a live Anthropic call during this recording, blockchain, or production-readiness — none of these are true of this build.
Concurrent guarantee explained correctly	The spoken numbers match what's actually on screen for your final take's split — not the numbers from an earlier rehearsal.
Evidence tamper failure visible	The red "INTEGRITY VIOLATION DETECTED" state and the named entry are both clearly readable, not cut too early.
Strong ending	Closing line lands, holds 2 seconds, no trailing dead air before the cut to black.
No accidental dead time	No long unedited waits, no visible typing of pre-stageable fields, no repeated explanations of the same UI label.